嵌入式设备免证书安全通信方案

https://gemini.google.com/app/5ab32f12b86b9186

*User prompt: 嵌入式产品通常都是动态IP地址。正如下面所示，如果开启 HTTPS 或 WSS，就需要 提供证书。这个证书如果是自签名的，在浏览器找开就会有警告。而且证书的管理也很复杂。 有没有好的办法，既能够提供 HTTP / WS 服务，又能跳过 证书，实现网络安全？ 最关键的现实约束是：**普通浏览器不会因为证书是设备自己生成的，就信任它。**自签名证书可以加密连接，但会触发证书警告；二维码、mDNS 名称或网页里的证书指纹，也不能自动把证书加入浏览器信任库。 我建议不要试图用一个方案覆盖所有部署场景，而是定义两种明确的产品模式： 1. 产品集成模式：使用浏览器可验证的 HTTPS 面向有域名、DNS 或设备管理基础设施的产品： 为设备分配稳定的域名，例如 device-id.example.com。 设备生成并保护自己的私钥，只提交 CSR；由产品的证书服务签发证书，避免把 CA 私钥放在设备或组件中。 用受信任 CA 签发的证书；本地访问时通过 split DNS 或产品网络的 DNS，将域名解析到设备的局域网地址。 证书续期、撤销、设备更换和恢复出厂后的身份变更都纳入产品管理流程。 这对浏览器用户最透明，但要求产品有域名与证书运维能力。 2. 独立配网模式：限制明文 HTTP 的用途和时间 对没有域名或证书基础设施的独立设备，我建议：通过物理操作开启短时配网窗口，使用设备独立的 SoftAP 密码保护接入。 配网窗口只提供必要的网络配置和配对操作，不开放 OTA、设备控制、凭据读取等高风险功能。 配网完成或超时后关闭 HTTP 配网入口。 日常管理若仍需普通浏览器直连，应要求产品另外提供可验证证书；不能把“用户忽略浏览器警告”当作正式信任方案。 如果要求独立设备在任何网络上都能通过普通浏览器直连，同时又要浏览器验证设备身份，那么必须让客户端预先信任某个 CA/设备身份，或引入可信域名与证书签发服务。没有这类信任锚，单靠设备端代码无法消除浏览器证书警告。 其他可选路径 受管企业环境：通过 MDM/企业 CA 将根证书部署到客户端，再为设备签发证书。适合 IT 管理环境，不适合作为普通消费者的首次使用要求。 配套原生 App：App 可固定设备证书或公钥，再通过内嵌页面提供界面；安全性和体验可控，但不再是纯浏览器访问。 云端代理：浏览器访问厂商可信域名，由设备主动建立到云端的安全连接。可绕开本地证书信任问题，但引入云依赖、账号体系和额外运维成本。*

Response: 在不使用 TLS 证书、仅依赖 `HTTP / WS` 的前提下，可以通过**应用层端到端加密（PAKE + AEAD）或WebRTC 数据通道**在动态 IP 上实现高强度安全，这也是主流家用路由器后台、Apple HomeKit 以及乐鑫 ESP-IDF 等框架绕过浏览器证书警告的通行做法。

方案一：HTTP/WS + 应用层加密隧道（PAKE + 对称加密）

这是最直接“跳过证书”的方案：利用 HTTP 仅用于加载前端页面，随后的所有 API 请求或 WebSocket 通信全部在应用层自行加密（把 TLS 的工作搬到 JS 和设备固件的 C 代码里完成）。

- **密钥协商（PAKE 协议）：** 采用 **SRP-6a** 或 **SPAKE2+**（Matter 协议同款）口令认证密钥交换算法。用户输入设备密码或机身 PIN 码后，双方在不传输密码明文或哈希的前提下，安全协商出一次性的高强度会话密钥（Session Key）。
- **数据通道加密（AEAD）：** 协商出密钥后，所有 WebSocket 帧或 HTTP 报文体均使用 **ChaCha20-Poly1305** 或 **AES-GCM** 进行认证加密，并附带递增的序列号（Nonce）防止重放攻击。
- **浏览器端的关键踩坑点：** 现代浏览器规定原生的 `window.crypto.subtle`（Web Crypto API）仅在安全上下文（HTTPS 或 `localhost`）可用，在 `[http://192.168.](http://192.168.)x.x` 下会被直接禁用。前端必须打包纯 JS 或 WebAssembly 密码学库（如 `@noble/ciphers`、`@noble/curves` 或 `libsodium.js`）。
- **安全边界评估：** 该方案能 100% 防御局域网被动抓包（Wi-Fi 嗅探）、密码窃听、报文篡改和重放攻击。唯一的理论短板是：如果攻击者在用户**首次加载** `index.html` **的瞬间**发起局域网 ARP 欺骗并篡改前端 JS 代码，就能植入后门；但在绝大多数家庭或受控局域网中，这已经达到了极高的实用安全水准。

方案二：云端 HTTPS 静态页面（PWA） + WebRTC DataChannel 局域网直连

如果你希望彻底消除方案一中“首次加载 HTTP 页面时 JS 可能被篡改”的风险，同时又不想给设备配置域名证书，可以利用浏览器的 **WebRTC 豁免机制**。

- **为什么不能直接连 WS：** 浏览器禁止在 `https://` 页面中直接发起明文 `http://` 或 `ws://` 请求（Mixed Content 混合内容拦截）。

- **WebRTC 如何合法绕过 CA 证书：** 浏览器原生的 `RTCPeerConnection` 强制使用 DTLS 加密，但**不要求 CA 机构签发的证书**，而是允许设备使用现场生成的自签名证书，仅通过比对信令（SDP）中的证书哈希指纹（`a=fingerprint:sha-256 ...`）完成认证，**浏览器绝不会弹出任何不安全警告**。
- **落地流程：** 用户访问厂商托管在公网的 `[https://portal.vendor.com](https://portal.vendor.com)`（开启 PWA Service Worker 缓存后，断网也能在浏览器离线打开），通过配网通道或轻量信令交换 SDP，随后浏览器与嵌入式设备（运行轻量级 `libdatachannel`）在局域网内直接建立点对点的 **WebRTC DataChannel**，在里面传输控制指令或类 HTTP/WS 数据。
- **方案优势：** 既有公网 HTTPS 保证前端 JS 代码绝对不被篡改，又实现了局域网动态 IP 的原生加密直连，且设备端完全不需要维护复杂的 CA 证书生命周期。

方案三：轻量级请求签名（HMAC + 动态挑战码）

如果嵌入式设备的算力或 RAM 极其受限，且传输的数据本身不敏感（例如只需要看传感器温度，但必须防止黑客伪造控制指令）：

- **挑战-响应防篡改：** 保持 JSON 明文传输，但每次连接时设备下发一个随机 `Nonce`（挑战码）。
- **逐包签名：** 前端利用用户密码派生出的密钥，结合 `Nonce`、时间戳和请求体计算 `HMAC-SHA256` 签名放入 Header。设备端仅需几 KB 代码验证 HMAC 即可确保指令未被篡改且不可重放。

落地选型建议

- **纯离线局域网独立设备（如路由器、工业网关、智能家居本地后台）：** 首选 **方案一（HTTP/WS + SPAKE2+/SRP + ChaCha20-Poly1305）**，嵌入式端可直接复用 `mbedTLS` 或 `libsodium`，用户体验最丝滑。
- **高安全要求且允许首次联网缓存前端的设备（如无线投屏器、NAS、IPC 摄像机）：** 考虑 **方案二（HTTPS PWA + WebRTC DataChannel）**，兼顾零警告与端到端加密闭环。

User prompt: 我使用 esp-idf 开发嵌入式设备，可以使用上面的机制吗?

Response: 完全可以，而且乐鑫在 ESP-IDF 中已经**原生内置了“方案一（PAKE + AEAD 应用层加密）”的完整实现**。你既可以直接调用官方现成的安全组件，也可以基于内置密码学库自己封装一套轻量的 WebSocket 加密通道。

方式一：直接复用 ESP-IDF 官方 `protocomm` 组件（最省心）

乐鑫在配网（`wifi_provisioning`）和本地控制（`esp_local_ctrl`）中为了解决局域网无证书安全问题，专门开发了 `protocomm` 组件，底层直接运行在明文 `esp_http_server` 之上：

- **Security 1 模式：** 采用 `X25519` 椭圆曲线密钥交换 + `AES-CTR` 对称加密，并支持通过 PoP（Proof of Possession，即机身 PIN 码或设备密码）防止未授权访问。
- **Security 2 模式：** 原生实现了 **SRP-6a 密钥协商 + AES-256-GCM 认证加密**（源码位于 ESP-IDF 的 `components/protocomm/src/security/security2.c`），自带防窃听、防篡改与递增 Nonce 防重放机制。
- **对接限制：** `protocomm` 协议默认使用 **Protobuf** 格式封装握手与业务报文；如果前端网页愿意引入 `protobuf.js`，设备端只需几十行代码即可启用工业级加密。

方式二：`esp_http_server` (WebSocket) + `mbedTLS` / `libsodium`（最灵活）

如果你希望前端直接传输易读的 **JSON 数据**，不想折腾 Protobuf，可以在 WebSocket 上自行封装一层加密：

- **开启原生 WebSocket：** 在 `idf.py menuconfig` 中打开 `CONFIG_ESP_HTTPD_WS_SUPPORT`，即可在同一个 HTTP 端口上通过 `httpd_ws_recv_frame` 和 `httpd_ws_send_frame` 收发加密二进制帧。

- **调用内置 mbedTLS（享硬件加速）：** ESP-IDF 默认包含 mbedTLS，且 ESP32 全系支持 AES 与 SHA 硬件加速。你可以直接调用 mbedtls_gcm_crypt_and_tag（AES-GCM）或 mbedtls_chachapoly_encrypt_and_tag 加解密，几乎不占 CPU 算力。
- **使用 libsodium 简化前后端对接：** 运行 idf.py add-dependency "espressif/libsodium" 引入官方维护的组件，前端网页引入同款的 libsodium-wrappers，前后端可用完全一致的算法接口完成密钥派生与加密。
- **提取 SRP-6a 源码：** 你甚至可以直接 include ESP-IDF 内置的 esp_srp.h（位于 protocomm 组件内）在自己的 WebSocket 握手阶段完成 SRP-6a 密码验证，无需手写复杂的数学运算。

方式三：在 ESP-IDF 上使用 WebRTC DataChannel（视硬件资源而定）

- **官方与开源支持：** 乐鑫已推出官方 esp_webrtc 组件，此外开源社区也有专为 ESP-IDF 移植的轻量级 WebRTC 库 libpeer。
- **硬件门槛：** WebRTC 涉及 ICE、DTLS 和 SCTP 协议栈，运行起来至少需要消耗 80KB 到 150KB 以上的 RAM。如果你的芯片是带 PSRAM 的 **ESP32-S3** 或 **ESP32-P4** 可以轻松跑通；如果是无 PSRAM 的普通 ESP32 或 ESP32-C3，强烈建议选择 **方式二（WebSocket + 应用层加密）**，RAM 开销仅需几 KB。

User prompt: 我想知道 Shelly Group 的产品是如何解决 TLS 加密需要证书这个问题的？它们没有这个问题吗？

Response: **Shelly 同样有这个问题，它也无法绕过浏览器的证书信任机制。**

Shelly 的产品（Gen2/Gen3/Gen4 同样基于乐鑫 ESP32 和 ESP32-C6 开发）之所以在日常使用中不会弹出浏览器证书警告，是因为它采取了非常务实的工程折中：**局域网内默认根本不开 HTTPS / WSS，而是直接走明文 HTTP / WS，并配合应用层的 SHA-256 挑战-响应认证来保障安全。**

具体来说，Shelly 通过三层架构解决了这个问题：

1. 局域网默认方案：明文 HTTP/WS + SHA-256 摘要认证（零警告）

出厂的 Shelly 设备不包含任何预烧录的 HTTPS 服务端证书。当你在浏览器输入 Shelly 的局域网动态 IP 时，访问的是明文 http://，前端页面通过 ws://ip/rpc 建立明文 WebSocket 连接并收发 JSON-RPC 指令。

为了防止局域网抓包窃取密码或伪造控制指令，Shelly 在 HTTP 和 WebSocket 的 JSON 载荷里实现了符合 **RFC 7616 标准的 SHA-256 Digest（摘要）挑战-响应机制**（即前面提到的“方案三”）：

1. **设备端不存明文密码：** 当用户设置本地密码时（调用 Shelly.SetAuth），前端只把预计算的哈希值 ha1 = SHA256("admin:" + realm + ":" + password) 发送给设备保存，其中 realm 是设备的唯一 ID（例如 shellypro4pm-f008d1d8b8b8）。
2. **下发随机挑战码（Nonce）：** 前端通过 WebSocket 发送控制指令时，若未携带认证信息，设备会返回一个 401 JSON 错误帧，里面包含设备生成的随机令牌 nonce 和 realm。
3. **动态签名防重放：** 前端生成客户端随机数 cnonce 和递增计数器 nc（Nonce Counter），在本地计算出动态签名：response = SHA256(ha1 : nonce : nc : cnonce : "auth" : ha2) 随后将包含该签名的 auth 对象附在每个 JSON-RPC 请求中发给设备。
4. **工程优势：**
   - **密码永不触网：** 局域网嗅探者只能看到随机变化的哈希签名，无法反推出密码，也无法把抓到的报文重放给设备（因为每次请求 nc 都在递增）。
   - **完全不依赖 window.crypto.subtle：** 即使在明文 http:// 下浏览器的原生加密 API 被禁用，前端用百余行纯 JS 代码就能轻松实现 SHA-256 运算，设备端 ESP32 验证一次 SHA-256 更是微秒级开销。

2. 局域网高安全方案：开放证书上传接口（由用户自己解决信任）

从 Gen3（固件 v1.8.0+ / v2.0.0+）开始，Shelly 为政企内网和极客用户开放了本地 Web UI 的 HTTPS 支持，但**把证书签发与信任的工作完全交给了用户自己**：

- **自带证书上传 RPC：** 用户可以通过 `Shelly.PutHTTPServerCert` 和 `Shelly.PutHTTPServerKey` 接口（或网页端的 `Settings → TLS Configuration`），将自己生成的证书和未加密私钥上传到设备中，重启后即可开启本地 HTTPS。
- **不能免俗的浏览器警告：** Shelly 官方文档明确提示，如果你上传的是自签名证书，浏览器依然会弹出安全警告。想要消除警告，用户必须自己在局域网搭建私有 CA（Private PKI），把根证书手动安装到电脑/手机的系统信任库里，并且在签发设备证书时把 Shelly 的 IP 地址或本地域名写进证书的 `SAN (Subject Alternative Name)` 扩展字段中（意味着你还必须在路由器上为它绑定静态 DHCP IP）。
- **硬件选型细节（供 ESP-IDF 借鉴）：** Shelly 官方文档特别强调，在嵌入式设备上强烈建议使用 **ECC P-256 (`prime256v1`)** 椭圆曲线密钥而非 RSA-2048，因为 ECC 密钥体积小、TLS 握手占用的 RAM 极少，且在搭载 ESP32-C6 的 Gen4 设备上支持硬件加速。

3. 区分“入站”与“出站”：把真正的 TLS 留给出站连接

Shelly 之所以宣传支持强大的 TLS 加密，主要是指**设备作为客户端（TLS Client）发起的出站连接**：

- **设备连云端 / MQTT / Webhook：** 当 Shelly 主动连接 `control.shelly.cloud`、用户的私有 MQTT Broker 或触发 HTTPS Webhook 时，Shelly 是客户端，只需要用内置根证书库（或用户上传的自定义 `ca.crt`）去校验服务器的证书即可，完全不存在浏览器告警问题。
- **远程无警告控制：** 普通用户如果想要全程加密且无警告的控制体验，直接使用官方 App 或访问 `[https://control.shelly.cloud](https://control.shelly.cloud)`，由浏览器验证 Shelly 云端域名的合法公网证书，再通过云端加密隧道下发指令。

对你的 ESP-IDF 项目的借鉴意义

如果你的嵌入式产品和 Shelly 类似——**局域网传输的数据本身并不是高度机密的隐私（只是开关状态、传感器数值、设备配置），核心诉求是“防止密码被嗅探、防止未授权控制与指令重放”**：

- 你完全可以采用 **Shelly 同款架构（明文 HTTP/WS + RFC 7616 SHA-256 Digest 认证）**。
- 相比于完整加密报文体的方案（PAKE + AES-GCM），Digest 认证的好处是**所有 JSON 报文在浏览器 F12 开发者工具里依然清晰可读**，极其方便前后端联调与第三方智能家居系统（如 Home Assistant）对接，同时彻底避开了 HTTPS 证书警告与动态 IP 管理的泥潭。

Sources

1. Setting up HTTPS on your Shelly device
2. HTTPS always assumed when enhanced_security enabled #179427
3. Authentication | Shelly Technical Documentation
4. Shelly TLS: Supported Certs and Config

User prompt: 请帮我设计一套仿照 Shelly 的 ESP-IDF WebSocket JSON-RPC SHA-256 Digest 认证报文格式及前后端校验代码。

Response: 这套基于 ESP-IDF 的 WebSocket JSON-RPC 认证方案完全复刻了 Shelly Gen2/Gen3 的核心机制，并在其基础上做了一个关键的安全加固（将 RPC 的 `method` 绑定到签名中，防止中间人篡改指令）。

一、 密码学签名公式与防重放机制

Shelly 的认证基于 RFC 7616 HTTP Digest SHA-256 标准，核心在于**设备端只存储预计算哈希 `HA1`，永远不接触明文密码**，且通过 `nonce`（服务端随机数）与 `nc`（递增计数器）实现防重放。

1. **密码哈希（`HA1`，配置密码时计算一次并存入 NVS）：**

$$\text{HA1} = \text{SHA256_Hex}(\text{username} + ":" + \text{realm} + ":" + \text{password})$$

   - `username`：固定为 `"admin"`。
   - `realm`：设备唯一标识（如使用 MAC 地址拼接的 `"myesp-a1b2c3"`），作为天然的“盐值（Salt）”，保证相同密码在不同设备上的 `HA1` 完全不同。

2. **动作摘要（`HA2`，防指令篡改）：**

   - **Shelly 原版做法：** 固定计算 `SHA256_Hex("dummy_method:dummy_uri")`。
   - **增强做法（推荐）：** 将当前请求的 `method` 纳入计算，防止局域网攻击者截获合法签名后将 `"Switch.GetStatus"` 替换为 `"Switch.Set"`：

$$\text{HA2} = \text{SHA256_Hex}(\text{"rpc:"} + \text{method})$$

3. **最终动态签名（`response`，每次请求动态计算）：**

$$\text{response} = \text{SHA256_Hex}(\text{HA1} + ":" + \text{nonce} + ":" + \text{nc} + ":" + \text{cnonce} + ":\text{auth:}" + \text{HA2})$$

   - `nonce`：ESP32 生成的 32 位随机挑战码（定期轮换）。
   - `nc`（Nonce Count）：客户端对该 `nonce` 的使用次数（1, 2, 3... 严格递增），ESP32 记录上一次的 `nc`，凡是小于或等于已记录值的请求一律拒绝，彻底阻断重放攻击。
   - `cnonce`：前端生成的随机整数，防止选择明文攻击。

二、 WebSocket JSON-RPC 报文交互格式

整个交互无需单独的“登录接口”，认证信息直接内嵌在业务 JSON-RPC 帧中，且缓存在有效期内无需每次都触发 401。

1. 首次请求（未携带 `auth` 或 `nonce` 已过期）

前端直接发送业务请求：

```
{
  "id": 1,
  "method": "Switch.Set",
  "params": { "id": 0, "on": true }
}
```

2. 设备返回 401 挑战帧（下发 `nonce` 与 `realm`）

ESP32 发现请求缺少有效 `auth`，拒绝执行并返回挑战参数：

```
{
  "id": 1,
  "src": "myesp-a1b2c3",
  "error": {
    "code": 401,
    "message": "Unauthorized",
    "auth": {
```

```
      "auth_type": "digest",
      "realm": "myesp-a1b2c3",
      "nonce": 2849103812,
      "nc": 1,
      "algorithm": "SHA-256"
    }
  }
}
```

3. 前端附加 auth 自动重发（以及后续免握手请求）

前端捕获 401 后提取 nonce，计算签名并重发 id: 1；之后的 id: 2、id: 3 请求直接复用该 nonce 并将 nc 递增为 2、3，实现**单次握手、持续通行**：

```
{
  "id": 1,
  "method": "Switch.Set",
  "params": { "id": 0, "on": true },
  "auth": {
    "realm": "myesp-a1b2c3",
    "username": "admin",
    "nonce": 2849103812,
    "cnonce": 94817263,
    "nc": 1,
    "algorithm": "SHA-256",
    "response": "7f83b1657ff1fc53b92dc18148a1d65dfc2d4b1fa3d677284addd200126d9069"
  }
}
```

三、 ESP-IDF 服务端完整实现（C 语言）

在 CMakeLists.txt 中确保引入 esp_http_server、json（cJSON）、mbedtls 和 esp_timer。同时在 idf.py menuconfig 中开启 CONFIG_ESP_HTTPD_WS_SUPPORT=y。

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>
#include "esp_http_server.h"
#include "esp_random.h"
#include "esp_timer.h"
#include "esp_log.h"
#include "mbedtls/sha256.h"
#include "cJSON.h"

static const char *TAG = "WS_RPC_AUTH";

#define DEVICE_REALM       "myesp-a1b2c3"
#define NONCE_EXPIRE_US    (3600ULL * 1000000ULL) // Nonce 有效期：1小时

// 演示用：预先算好的 HA1 = SHA256("admin:myesp-a1b2c3:123456")
// 实际工程中请从 NVS 读取此 64 字节 Hex 字符串，切勿存明文密码
static char s_stored_ha1[65] = "6b50467780f6823941122e963c18c982d0626f137927a0b6c7b4059d7041c09e";

typedef struct {
    uint32_t active_nonce;
    uint32_t last_nc;
    int64_t  created_at_us;
} auth_session_t;

static auth_session_t s_auth_ctx = {0};

// 工具函数：计算输入字符串的 SHA-256 并输出 64 字符小写 Hex
static void sha256_hex(const char *input, char output_hex[65])
{
    unsigned char hash[32];
    mbedtls_sha256_context ctx;
```

```
    mbedtls_sha256_init(&ctx);
    mbedtls_sha256_starts(&ctx, 0); // 0 表示 SHA-256
    mbedtls_sha256_update(&ctx, (const unsigned char *)input, strlen(input));
    mbedtls_sha256_finish(&ctx, hash);
    mbedtls_sha256_free(&ctx);

    for (int i = 0; i < 32; i++) {
        sprintf(output_hex + (i * 2), "%02x", hash[i]);
    }
    output_hex[64] = '\0';
}

// 刷新或初始化服务端 Nonce
static void refresh_nonce_if_needed(bool force)
{
    int64_t now = esp_timer_get_time();
    if (force || s_auth_ctx.active_nonce == 0 || (now - s_auth_ctx.created_at_us) > NONCE_EXPIRE_US) {
        s_auth_ctx.active_nonce = esp_random();
        if (s_auth_ctx.active_nonce == 0) s_auth_ctx.active_nonce = 1;
        s_auth_ctx.last_nc = 0;
        s_auth_ctx.created_at_us = now;
        ESP_LOGI(TAG, "Generated new nonce: %lu", (unsigned long)s_auth_ctx.active_nonce);
    }
}

// 校验 JSON-RPC 中的 auth 字段
static bool verify_rpc_auth(cJSON *auth_obj, const char *method)
{
    if (!auth_obj || !method) return false;

    cJSON *j_nonce    = cJSON_GetObjectItem(auth_obj, "nonce");
    cJSON *j_cnonce   = cJSON_GetObjectItem(auth_obj, "cnonce");
    cJSON *j_nc       = cJSON_GetObjectItem(auth_obj, "nc");
    cJSON *j_response = cJSON_GetObjectItem(auth_obj, "response");

    if (!cJSON_IsNumber(j_nonce) || !cJSON_IsNumber(j_cnonce) ||
        !cJSON_IsNumber(j_nc) || !cJSON_IsString(j_response)) {
        return false;
    }

    refresh_nonce_if_needed(false);

    uint32_t client_nonce  = (uint32_t)j_nonce->valuedouble;
    uint32_t client_cnonce = (uint32_t)j_cnonce->valuedouble;
    uint32_t client_nc     = (uint32_t)j_nc->valuedouble;

    // 1. 校验 Nonce 是否匹配，以及 nc 是否严格递增（防重放核心）
    if (client_nonce != s_auth_ctx.active_nonce) {
        ESP_LOGW(TAG, "Nonce mismatch: got %lu, expected %lu",
                 (unsigned long)client_nonce, (unsigned long)s_auth_ctx.active_nonce);
        return false;
    }
    if (client_nc <= s_auth_ctx.last_nc) {
        ESP_LOGW(TAG, "Replay detected! client_nc=%lu <= last_nc=%lu",
                 (unsigned long)client_nc, (unsigned long)s_auth_ctx.last_nc);
        return false;
    }

    // 2. 计算 HA2 = SHA256("rpc:" + method)
    char ha2_input[128];
    char ha2_hex[65];
    snprintf(ha2_input, sizeof(ha2_input), "rpc:%s", method);
    sha256_hex(ha2_input, ha2_hex);

    // 3. 计算期望的 response = SHA256(HA1:nonce:nc:cnonce:auth:HA2)
    char resp_input[256];
    char expected_resp[65];
    snprintf(resp_input, sizeof(resp_input), "%s:%lu:%lu:%lu:auth:%s",
             s_stored_ha1,
```

```
                (unsigned long)client_nonce,
                (unsigned long)client_nc,
                (unsigned long)client_cnonce,
                ha2_hex);
    sha256_hex(resp_input, expected_resp);

    // 4. 恒定时间比对签名
    if (strncmp(expected_resp, j_response->valuestring, 64) != 0) {
        ESP_LOGW(TAG, "Invalid signature");
        return false;
    }

    // 验证通过，更新防重放水位线
    s_auth_ctx.last_nc = client_nc;
    return true;
}

// 发送 JSON 响应帧的辅助函数
static esp_err_t ws_send_json(httpd_req_t *req, cJSON *root)
{
    char *json_str = cJSON_PrintUnformatted(root);
    if (!json_str) return ESP_ERR_NO_MEM;

    httpd_ws_frame_t ws_pkt = {
        .final = true,
        .fragmented = false,
        .type = HTTPD_WS_TYPE_TEXT,
        .payload = (uint8_t *)json_str,
        .len = strlen(json_str)
    };
    esp_err_t ret = httpd_ws_send_frame(req, &ws_pkt);
    free(json_str);
    return ret;
}

// WebSocket /rpc 路由处理函数
static esp_err_t ws_rpc_handler(httpd_req_t *req)
{
    if (req->method == HTTP_GET) {
        ESP_LOGI(TAG, "WebSocket handshake done");
        return ESP_OK;
    }

    httpd_ws_frame_t ws_pkt = {0};
    ws_pkt.type = HTTPD_WS_TYPE_TEXT;

    // 先获取帧长度
    esp_err_t ret = httpd_ws_recv_frame(req, &ws_pkt, 0);
    if (ret != ESP_OK || ws_pkt.len == 0) return ret;

    uint8_t *buf = calloc(1, ws_pkt.len + 1);
    if (!buf) return ESP_ERR_NO_MEM;
    ws_pkt.payload = buf;

    ret = httpd_ws_recv_frame(req, &ws_pkt, ws_pkt.len);
    if (ret != ESP_OK) {
        free(buf);
        return ret;
    }

    cJSON *req_json = cJSON_Parse((char *)buf);
    free(buf);
    if (!req_json) return ESP_FAIL;

    cJSON *j_id     = cJSON_GetObjectItem(req_json, "id");
    cJSON *j_method = cJSON_GetObjectItem(req_json, "method");
    cJSON *j_auth   = cJSON_GetObjectItem(req_json, "auth");
    int req_id      = cJSON_IsNumber(j_id) ? j_id->valueint : 0;
    const char *method_str = cJSON_IsString(j_method) ? j_method->valuestring : "";
```

```
    cJSON *resp = cJSON_CreateObject();
    cJSON_AddNumberToObject(resp, "id", req_id);
    cJSON_AddStringToObject(resp, "src", DEVICE_REALM);

    // 执行 Digest 校验
    if (!verify_rpc_auth(j_auth, method_str)) {
        refresh_nonce_if_needed(false);

        cJSON *err_obj = cJSON_AddObjectToObject(resp, "error");
        cJSON_AddNumberToObject(err_obj, "code", 401);
        cJSON_AddStringToObject(err_obj, "message", "Unauthorized");

        cJSON *challenge = cJSON_AddObjectToObject(err_obj, "auth");
        cJSON_AddStringToObject(challenge, "auth_type", "digest");
        cJSON_AddStringToObject(challenge, "realm", DEVICE_REALM);
        cJSON_AddNumberToObject(challenge, "nonce", (double)s_auth_ctx.active_nonce);
        cJSON_AddNumberToObject(challenge, "nc", (double)(s_auth_ctx.last_nc + 1));
        cJSON_AddStringToObject(challenge, "algorithm", "SHA-256");
    } else {
        // 认证通过，执行实际业务 RPC（示例：Switch.Set）
        cJSON *result = cJSON_AddObjectToObject(resp, "result");
        cJSON_AddStringToObject(result, "status", "ok");
        cJSON_AddStringToObject(result, "executed_method", method_str);
    }

    ret = ws_send_json(req, resp);
    cJSON_Delete(resp);
    cJSON_Delete(req_json);
    return ret;
}
```

四、 浏览器前端完整实现（纯 JS，无需 crypto.subtle）

由于局域网 http://192.168.x.x 下浏览器会禁用 window.crypto.subtle，以下代码内置了一个轻量级的纯 JavaScript SHA-256 实现，并封装了支持**自动捕获 401 挑战、自动重发、自动递增** nc 的 WebSocket RPC 客户端：

```
// 1. 纯 JS SHA-256 实现（兼容非 HTTPS 局域网环境）
function sha256Hex(ascii) {
  function rightRotate(value, amount) {
    return (value >>> amount) | (value << (32 - amount));
  }
  const mathPow = Math.pow, maxWord = mathPow(2, 32);
  let result = '';
  const words = [], asciiBitLength = ascii.length * 8;
  let hash = sha256Hex.h = sha256Hex.h || [];
  const k = sha256Hex.k = sha256Hex.k || [];
  let primeCounter = k.length;
  const isComposite = {};
  for (let candidate = 2; primeCounter < 64; candidate++) {
    if (!isComposite[candidate]) {
      for (let i = 0; i < 313; i += candidate) isComposite[i] = candidate;
      hash[primeCounter] = (mathPow(candidate, 0.5) * maxWord) | 0;
      k[primeCounter++] = (mathPow(candidate, 1 / 3) * maxWord) | 0;
    }
  }
  ascii += '\x80';
  while ((ascii.length % 64) - 56) ascii += '\x00';
  for (let i = 0; i < ascii.length; i++) {
    const j = ascii.charCodeAt(i);
    words[i >> 2] |= j << ((3 - (i % 4)) * 8);
  }
  words[words.length] = (asciiBitLength / maxWord) | 0;
  words[words.length] = asciiBitLength;
  for (let j = 0; j < words.length; ) {
    const w = words.slice(j, (j += 16)), oldHash = hash;
```

```
      hash = hash.slice(0, 8);
      for (let i = 0; i < 64; i++) {
        const i2 = i + j, w15 = w[i - 15], w2 = w[i - 2];
        const a = hash[0], e = hash[4];
        const temp1 = hash[7] + (rightRotate(e, 6) ^ rightRotate(e, 11) ^ rightRotate(e, 25)) +
          ((e & hash[5]) ^ (~e & hash[6])) + k[i] +
          (w[i] = i < 16 ? w[i] : (w[i - 16] + (rightRotate(w15, 7) ^ rightRotate(w15, 18) ^ (w15 >>> 3)) +
            w[i - 7] + (rightRotate(w2, 17) ^ rightRotate(w2, 19) ^ (w2 >>> 10))) | 0);
        const temp2 = (rightRotate(a, 2) ^ rightRotate(a, 13) ^ rightRotate(a, 22)) +
          ((a & hash[1]) ^ (a & hash[2]) ^ (hash[1] & hash[2]));
        hash = [(temp1 + temp2) | 0].concat(hash);
        hash[4] = (hash[4] + temp1) | 0;
      }
      for (let i = 0; i < 8; i++) hash[i] = (hash[i] + oldHash[i]) | 0;
    }
    for (let i = 0; i < 8; i++) {
      for (let j = 3; j + 1; j--) {
        const b = (hash[i] >> (j * 8)) & 255;
        result += (b < 16 ? 0 : '') + b.toString(16);
      }
    }
    return result;
  }

  // 2. 仿 Shelly WebSocket RPC 客户端
  class EspRpcClient {
    constructor(wsUrl, password) {
      this.ws = new WebSocket(wsUrl);
      this.password = password;
      this.seqId = 1;
      this.pending = new Map();
      this.authCache = null; // 缓存 { realm, nonce, nc, ha1 }

      this.ws.onmessage = (event) => this._handleMessage(JSON.parse(event.data));
    }

    // 构建 Digest Auth 对象
    _buildAuth(method) {
      if (!this.authCache) return undefined;
      const { realm, nonce, ha1 } = this.authCache;
      const nc = this.authCache.nc++;
      const cnonce = Math.floor(Math.random() * 0xffffffff);
      const ha2 = sha256Hex(`rpc:${method}`);
      const response = sha256Hex(`${ha1}:${nonce}:${nc}:${cnonce}:auth:${ha2}`);

      return {
        realm,
        username: 'admin',
        nonce,
        cnonce,
        nc,
        algorithm: 'SHA-256',
        response
      };
    }

    // 发送 RPC 请求
    call(method, params = {}) {
      return new Promise((resolve, reject) => {
        const id = this.seqId++;
        const payload = { id, method, params };
        if (this.authCache) {
          payload.auth = this._buildAuth(method);
        }
        this.pending.set(id, { method, params, retried: false, resolve, reject });
        this.ws.send(JSON.stringify(payload));
      });
    }
```

```
  // 处理响应与 401 自动重试
  _handleMessage(msg) {
    const req = this.pending.get(msg.id);
    if (!req) return;

    // 捕获 401 挑战并自动签名重发
    if (msg.error && msg.error.code === 401 && msg.error.auth && !req.retried) {
      const { realm, nonce, nc } = msg.error.auth;
      const ha1 = sha256Hex(`admin:${realm}:${this.password}`);
      this.authCache = { realm, nonce, nc: nc || 1, ha1 };

      req.retried = true;
      const retryPayload = {
        id: msg.id,
        method: req.method,
        params: req.params,
        auth: this._buildAuth(req.method)
      };
      this.ws.send(JSON.stringify(retryPayload));
      return;
    }

    this.pending.delete(msg.id);
    if (msg.error) {
      req.reject(msg.error);
    } else {
      req.resolve(msg.result);
    }
  }
}

// 使用示例：
// const client = new EspRpcClient('ws://192.168.1.50/rpc', '123456');
// client.ws.onopen = async () => {
//   const res = await client.call('Switch.Set', { id: 0, on: true });
//   console.log('执行成功:', res);
// };
```

五、 工程落地的三个安全细节

1. **密码修改接口（`Auth.SetPassword`）设计：** 当用户在前端修改密码时，前端必须在本地算出新的 `ha1 = sha256Hex("admin:" + realm + ":" + newPassword)`，并将 `ha1` 作为参数通过已认证的 RPC 发送给设备。设备切勿提供接收明文密码的接口。
2. **多客户端并发连接（手机 + 电脑同时打开网页）：** 上面的示例使用全局单例 `s_auth_ctx`。如果允许多个浏览器同时连接同一个 ESP32，建议在 `ws_rpc_handler` 中利用 `req->sess_ctx` 为每个 WebSocket 连接分配独立的 `auth_session_t` 结构体，避免多个客户端之间的 `nc` 计数器互相覆盖冲突。
3. **暴力破解防护：** ESP32 计算一次 SHA-256 仅需几微秒，为防止局域网攻击者编写脚本高频穷举短密码，当 `verify_rpc_auth` 连续失败超过 5 次（且不是因为首次无 `auth` 触发的挑战）时，应强制调用 `vTaskDelay(pdMS_TO_TICKS(2000))` 或在 30 秒内拒绝该 Socket 的认证请求。

sess_ctx）与密码修改存入 NVS 的完整 C 代码实现吗？" query="请帮我补充基于 esp_http_server req->sess_ctx 的多客户端独立会话管理，以及通过 RPC 修改 HA1 密码并持久化到 NVS 的完整 ESP-IDF 代码。"/>
